01 — Model Training Tournament (GTZAN)

Esegue il torneo su 3 architetture e implementa WP1 (ablation augmentation) e WP2 (efficienza).

In [ ]:
```
# Setup & data load
import os, pickle, time, random, numpy as np, pandas as pd, tensorflow as tf
import keras
from keras import layers, models, callbacks
from keras.utils import to_categorical
from pathlib import Path

# Lightweight logger
VERBOSE = int(os.getenv('GTZAN_VERBOSE', '1'))

def log(msg: str, level: str = 'INFO'):
    if VERBOSE:
        ts = time.strftime('%H:%M:%S')
        print(f"[{ts}] {level}: {msg}")

# --- Reproducibility ---
RANDOM_STATE = 42
random.seed(RANDOM_STATE)
np.random.seed(RANDOM_STATE)
tf.random.set_seed(RANDOM_STATE)

# --- GPU config & Mixed Precision ---
gpus = tf.config.list_physical_devices('GPU')
if gpus:
    try:
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
        log(f"GPU(s) detected: {[tf.config.experimental.get_device_details(g
    except Exception as e:
        log(f"Could not set memory growth: {e}", level='WARN')
try:
    policy = keras.mixed_precision.Policy('mixed_float16')
    keras.mixed_precision.set_global_policy(policy)
    log(f"Mixed precision enabled: {keras.mixed_precision.global_policy().na
except Exception as e:
    log(f"Could not enable mixed precision: {e}", level='WARN')

PROJECT_ROOT = Path(os.getcwd()).resolve().parents[1]
PROCESSED = PROJECT_ROOT/'data'/'processed'
MODELS = PROJECT_ROOT/'models'
REPORTS = PROJECT_ROOT/'reports'
MODELS.mkdir(exist_ok=True); REPORTS.mkdir(exist_ok=True)

X_train = np.load(PROCESSED/'X_train.npy'); y_train = np.load(PROCESSED/'y_t
X_val = np.load(PROCESSED/'X_val.npy'); y_val = np.load(PROCESSED/'y_val.npy
```

```
X_test = np.load(PROCESSED/'X_test.npy'); y_test = np.load(PROCESSED/'y_test
# Ensure float32 inputs for efficient training
X_train = X_train.astype(np.float32)
X_val = X_val.astype(np.float32)
X_test = X_test.astype(np.float32)

with open(PROCESSED/'label_encoder.pkl','rb') as f: le = pickle.load(f)
num_classes = len(le.classes_)

y_train_cat = to_categorical(y_train, num_classes)
y_val_cat = to_categorical(y_val, num_classes)
y_test_cat = to_categorical(y_test, num_classes)

log(f"Shapes: train={X_train.shape} val={X_val.shape} test={X_test.shape} |
```

```
Warning: could not set memory growth: Physical devices cannot be modified af
ter being initialized
Mixed precision enabled: mixed_float16
Shapes: (6000, 128, 128, 1) (2000, 128, 128, 1) (2000, 128, 128, 1) | classe
s: 10
```

In [ ]:

```
# SpecAugment toggleable
@tf.function
def spec_augment_tf(s, y):
    """
    Apply SpecAugment-style frequency and time masking to a single spectrogr

    @param s: Spectrogram tensor with shape [freq_bins, time_steps, channels
    @param y: One-hot label tensor corresponding to s.
    @return: Tuple (augmented_s, y).
    """
    # Avoid direct tensor indexing on shapes; use tf.shape with tf.gather.
    s_shape = tf.shape(s)
    freq_bins = tf.gather(s_shape, 0)
    time_steps = tf.gather(s_shape, 1)

    # Frequency mask
    max_f = tf.maximum(
        tf.constant(2, dtype=tf.int32),
        tf.cast(0.2 * tf.cast(freq_bins, tf.float32), tf.int32),
    )
    f = tf.random.uniform([], minval=1, maxval=max_f, dtype=tf.int32)
    max_f0 = tf.maximum(tf.constant(1, dtype=tf.int32), freq_bins - f)
    f0 = tf.random.uniform([], minval=0, maxval=max_f0, dtype=tf.int32)
    mask_f = tf.concat(
        [
            tf.ones([f0], dtype=s.dtype),
            tf.zeros([f], dtype=s.dtype),
            tf.ones([freq_bins - f0 - f], dtype=s.dtype),
        ],
        axis=0,
    )
    s = s * tf.reshape(mask_f, [freq_bins, 1, 1])

    # Time mask
    max_t = tf.maximum(
```

```
        tf.constant(2, dtype=tf.int32),
        tf.cast(0.2 * tf.cast(time_steps, tf.float32), tf.int32),
    )
    t = tf.random.uniform([], minval=1, maxval=max_t, dtype=tf.int32)
    max_t0 = tf.maximum(tf.constant(1, dtype=tf.int32), time_steps - t)
    t0 = tf.random.uniform([], minval=0, maxval=max_t0, dtype=tf.int32)
    mask_t = tf.concat(
        [
            tf.ones([t0], dtype=s.dtype),
            tf.zeros([t], dtype=s.dtype),
            tf.ones([time_steps - t0 - t], dtype=s.dtype),
        ],
        axis=0,
    )
    s = s * tf.reshape(mask_t, [1, time_steps, 1])

    return s, y


def make_pipelines(use_aug: bool, batch=48):
    """
    Build tf.data pipelines for training, validation, and test sets.

    @param use_aug: If True, apply SpecAugment on-the-fly to the training se
    @param batch: Batch size for all datasets.
    @return: Tuple (train_ds, val_ds, test_ds).
    """
    AUTOTUNE = tf.data.AUTOTUNE

    # Training dataset: shuffle deterministically and apply augmentation onl
    ds_train = (
        tf.data.Dataset.from_tensor_slices((X_train, y_train_cat))
        .shuffle(buffer_size=len(X_train), seed=RANDOM_STATE, reshuffle_each
    )
    if use_aug:
        ds_train = ds_train.map(spec_augment_tf, num_parallel_calls=AUTOTUNE
    ds_train = ds_train.batch(batch, drop_remainder=True).prefetch(AUTOTUNE)

    # Validation/Test: cache and prefetch for speed. No augmentation.
    ds_val = (
        tf.data.Dataset.from_tensor_slices((X_val, y_val_cat))
        .batch(batch)
        .cache()
        .prefetch(AUTOTUNE)
    )
    ds_test = (
        tf.data.Dataset.from_tensor_slices((X_test, y_test_cat))
        .batch(batch)
        .cache()
        .prefetch(AUTOTUNE)
    )
    return ds_train, ds_val, ds_test
```

In [ ]:
```
# Curated ModelFactory (aligned with original curated definitions)
from keras import layers, models
```

```python
class ModelFactory:
    """
    A curated factory for building and comparing the three key CNN
    architectures selected for our final, focused analysis.
    """

    # --------------------------- Helper blocks ---------------------------
    @staticmethod
    def _se_block(input_tensor, ratio=8, name_prefix=""):
        """Squeeze-and-Excitation block to add channel-wise attention."""
        channels = input_tensor.shape[-1]
        se = layers.GlobalAveragePooling2D(name=f'{name_prefix}_se_squeeze')
        se = layers.Reshape((1, 1, channels))(se)
        se = layers.Dense(channels // ratio, activation='relu', name=f'{name
        se = layers.Dense(channels, activation='sigmoid', name=f'{name_prefi
        return layers.Multiply(name=f'{name_prefix}_se_scale')([input_tensor

    # --------------------------- Efficient VGG ---------------------------
    @staticmethod
    def build_efficient_vgg(input_shape, num_classes):
        inputs = layers.Input(shape=input_shape)
        # Block 1
        x = layers.Conv2D(16, 3, padding='same', use_bias=False)(inputs)
        x = layers.BatchNormalization()(x); x = layers.Activation('relu')(x)
        x = layers.MaxPooling2D(2)(x)
        x = ModelFactory._se_block(x, name_prefix="vgg_b1")
        # Block 2
        x = layers.Conv2D(32, 3, padding='same', use_bias=False)(x)
        x = layers.BatchNormalization()(x); x = layers.Activation('relu')(x)
        x = layers.MaxPooling2D(2)(x)
        x = ModelFactory._se_block(x, name_prefix="vgg_b2")
        # Block 3
        x = layers.Conv2D(64, 3, padding='same', use_bias=False)(x)
        x = layers.BatchNormalization()(x); x = layers.Activation('relu')(x)
        x = layers.MaxPooling2D(2)(x)
        x = ModelFactory._se_block(x, name_prefix="vgg_b3")
        # Head
        x = layers.GlobalAveragePooling2D(name="gap")(x)
        x = layers.Dense(128, activation='relu')(x)
        x = layers.Dropout(0.5)(x)
        outputs = layers.Dense(num_classes, activation='softmax', dtype='flo
        return models.Model(inputs=inputs, outputs=outputs, name='Efficient_

    # --------------------------- ResSE AudioCNN ---------------------------
    @staticmethod
    def _res_se_block(input_tensor, filters, stride=1, name_prefix=""):
        shortcut = input_tensor
        x = layers.Conv2D(filters, 3, strides=stride, padding='same', use_bi
        x = layers.BatchNormalization(name=f'{name_prefix}_bn1')(x)
        x = layers.PReLU(shared_axes=[1, 2], name=f'{name_prefix}_prelu1')(x
        x = layers.Conv2D(filters, 3, padding='same', use_bias=False, name=f
        x = layers.BatchNormalization(name=f'{name_prefix}_bn2')(x)
        x = ModelFactory._se_block(x, name_prefix=f'{name_prefix}_se')
        if stride > 1 or shortcut.shape[-1] != filters:
            shortcut = layers.Conv2D(filters, 1, strides=stride, use_bias=Fa
            shortcut = layers.BatchNormalization(name=f'{name_prefix}_shortc
```

```
        x = layers.Add(name=f'{name_prefix}_add')([shortcut, x])
        x = layers.PReLU(shared_axes=[1, 2], name=f'{name_prefix}_prelu2')(x
        return x

    @staticmethod
    def build_res_se_audio_cnn(input_shape, num_classes):
        inputs = layers.Input(shape=input_shape)
        x = layers.Conv2D(32, 3, strides=1, padding='same', use_bias=False)(
        x = layers.BatchNormalization()(x)
        x = layers.PReLU(shared_axes=[1, 2])(x)
        x = ModelFactory._res_se_block(x, 64, stride=2, name_prefix="res_b1"
        x = ModelFactory._res_se_block(x, 128, stride=2, name_prefix="res_b2
        x = ModelFactory._res_se_block(x, 256, stride=2, name_prefix="res_b3
        x = layers.GlobalAveragePooling2D(name="gap")(x)
        x = layers.Dropout(0.5)(x)
        outputs = layers.Dense(num_classes, activation='softmax', dtype='flo
        return models.Model(inputs=inputs, outputs=outputs, name='ResSE_Audi

    # ---------------------------- UNet Audio Classifier ------------------
    @staticmethod
    def _unet_encoder_block(input_tensor, filters, pool=True, name_prefix=""
        x = layers.Conv2D(filters, 3, padding='same', use_bias=False, name=f
        x = layers.BatchNormalization(name=f'{name_prefix}_bn1')(x); x = lay
        x = layers.Conv2D(filters, 3, padding='same', use_bias=False, name=f
        x = layers.BatchNormalization(name=f'{name_prefix}_bn2')(x); x = lay
        skip_connection = x
        if pool:
            pool_output = layers.MaxPooling2D(2, name=f'{name_prefix}_pool')
            return pool_output, skip_connection
        else:
            return x, skip_connection

    @staticmethod
    def build_unet_audio_classifier(input_shape, num_classes):
        inputs = layers.Input(shape=input_shape)
        # Encoder path
        p1, s1 = ModelFactory._unet_encoder_block(inputs, 32, name_prefix="e
        p2, s2 = ModelFactory._unet_encoder_block(p1, 64, name_prefix="enc2"
        p3, s3 = ModelFactory._unet_encoder_block(p2, 128, name_prefix="enc3
        # Bottleneck (pool=False)
        bottleneck, _ = ModelFactory._unet_encoder_block(p3, 256, pool=False
        # Classification head
        x = layers.GlobalAveragePooling2D(name="gap")(bottleneck)
        x = layers.Dropout(0.5)(x)
        outputs = layers.Dense(num_classes, activation='softmax', dtype='flo
        return models.Model(inputs=inputs, outputs=outputs, name='UNet_Audio

# Final registry of models
FINAL_MODELS = {
    'Efficient_VGG': ModelFactory.build_efficient_vgg,
    'ResSE_AudioCNN': ModelFactory.build_res_se_audio_cnn,
    'UNet_Audio_Classifier': ModelFactory.build_unet_audio_classifier,
}
```

In [ ]:
```
# Training/eval orchestration with efficiency metrics (WP2)
```

```python
def count_params(model):
    return np.sum([np.prod(v.shape) for v in model.trainable_variables])


def measure_latency(model, sample, runs=50, warmup=5):
    x = tf.convert_to_tensor(sample)
    for _ in range(warmup):
        _ = model(x, training=False)
    start = time.time()
    for _ in range(runs):
        _ = model(x, training=False)
    end = time.time()
    return (end - start) / runs * 1000.0


def approximate_flops(model):
    """
    Rough FLOPs estimator (multiply-adds) for common layers.
    - Conv2D: H*W*C_in*C_out*K*K*2
    - DepthwiseConv2D: H*W*C_out*K*K*2
    - Dense: in*out*2
    Returns total FLOPs for a single forward pass on batch size 1.
    """
    total = 0
    from keras.layers import Conv2D, DepthwiseConv2D, Dense, Conv2DTranspose

    for layer in model.layers:
        try:
            out_shape = layer.output_shape
        except Exception:
            continue
        if isinstance(layer, Conv2D):
            if None in (out_shape[1], out_shape[2], out_shape[3]):
                continue
            k_h, k_w = layer.kernel_size
            H, W, C_out = out_shape[1], out_shape[2], out_shape[3]
            C_in = layer.input_shape[-1]
            total += H * W * C_in * C_out * k_h * k_w * 2
        elif isinstance(layer, DepthwiseConv2D):
            if None in (out_shape[1], out_shape[2], out_shape[3]):
                continue
            k_h, k_w = layer.kernel_size
            H, W, C_out = out_shape[1], out_shape[2], out_shape[3]
            total += H * W * C_out * k_h * k_w * 2
        elif isinstance(layer, Conv2DTranspose):
            if None in (out_shape[1], out_shape[2], out_shape[3]):
                continue
            k_h, k_w = layer.kernel_size
            H, W, C_out = out_shape[1], out_shape[2], out_shape[3]
            C_in = layer.input_shape[-1]
            total += H * W * C_in * C_out * k_h * k_w * 2
        elif isinstance(layer, Dense):
            try:
                in_units = layer.input_shape[-1]
                out_units = layer.units
                if None not in (in_units, out_units):
```

```
                    total += in_units * out_units * 2
            except Exception:
                pass
    return float(total)


def run_tournament(use_aug: bool, tag: str, epochs=120, patience=25, batch=4
    train_ds, val_ds, test_ds = make_pipelines(use_aug, batch)
    input_shape = X_train.shape[1:]
    results = []
    for name, builder in FINAL_MODELS.items():
        keras.backend.clear_session()
        model = builder(input_shape, num_classes)
        model.compile(optimizer=keras.optimizers.Adam(1e-3), loss='categoric
        ckpt_path = MODELS / f"{name}_best_{tag}.keras"
        cb = [
            callbacks.EarlyStopping(monitor='val_accuracy', patience=patienc
            callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.2, pati
            callbacks.ModelCheckpoint(ckpt_path, monitor='val_accuracy', sav
        ]
        log(f"Training {name} [{tag}] for up to {epochs} epochs...")
        h = model.fit(train_ds, validation_data=val_ds, epochs=epochs, verbo
        test_loss, test_acc = model.evaluate(test_ds, verbose=0)
        params = count_params(model)
        sample = X_test[:1]
        latency_ms = measure_latency(model, sample)
        flops = approximate_flops(model)
        best_val = float(np.max(h.history.get('val_accuracy', [0])))
        log(f"{name} [{tag}] -> Best Val: {best_val:.4f} | Test: {float(test
        results.append({
            'Model': name,
            'Tag': tag,
            'Best_Val_Accuracy': best_val,
            'Test_Accuracy': float(test_acc),
            'Epochs_Run': int(len(h.history.get('val_accuracy', []))),
            'Params': int(params),
            'Approx_FLOPs': float(flops) if np.isfinite(flops) else None,
            'Latency_ms': float(latency_ms),
        })
    df = pd.DataFrame(results)
    out_csv = REPORTS / f"training_summary_{tag}.csv"
    df.to_csv(out_csv, index=False)
    log(f'Saved: {out_csv}')
    return df
```

In [ ]:

```
# Run WP1: WITH_AUG and NO_AUG, and display WP2 metrics
df_with = run_tournament(use_aug=True, tag='WITH_AUG')
df_no = run_tournament(use_aug=False, tag='NO_AUG')

from IPython.display import display
display(pd.concat([df_with, df_no], ignore_index=True))
```

```
Training Efficient_VGG [WITH_AUG] for up to 120 epochs...
Efficient_VGG [WITH_AUG] -> Best Val: 0.7530 | Test: 0.7520
Efficient_VGG [WITH_AUG] -> Best Val: 0.7530 | Test: 0.7520

Training ResSE_AudioCNN [WITH_AUG] for up to 120 epochs...

Training ResSE_AudioCNN [WITH_AUG] for up to 120 epochs...
2025-08-17 11:05:43.930991: I external/local_xla/xla/stream_executor/cuda/su
bprocess_compilation.cc:346] ptxas warning : Registers are spilled to local
memory in function 'gemm_fusion_dot_5', 8 bytes spill stores, 8 bytes spill
loads

2025-08-17 11:05:52.909217: I external/local_xla/xla/stream_executor/cuda/su
bprocess_compilation.cc:346] ptxas warning : Registers are spilled to local
memory in function 'gemm_fusion_dot_5', 8 bytes spill stores, 8 bytes spill
loads

2025-08-17 11:05:52.909217: I external/local_xla/xla/stream_executor/cuda/su
bprocess_compilation.cc:346] ptxas warning : Registers are spilled to local
memory in function 'gemm_fusion_dot_5', 8 bytes spill stores, 8 bytes spill
loads

ResSE_AudioCNN [WITH_AUG] -> Best Val: 0.8155 | Test: 0.7980

Training UNet_Audio_Classifier [WITH_AUG] for up to 120 epochs...

Training UNet_Audio_Classifier [WITH_AUG] for up to 120 epochs...
UNet_Audio_Classifier [WITH_AUG] -> Best Val: 0.8525 | Test: 0.8230
Saved: /home/alepot55/Desktop/projects/naml_project/reports/training_summary
_WITH_AUG.csv
UNet_Audio_Classifier [WITH_AUG] -> Best Val: 0.8525 | Test: 0.8230
Saved: /home/alepot55/Desktop/projects/naml_project/reports/training_summary
_WITH_AUG.csv

Training Efficient_VGG [NO_AUG] for up to 120 epochs...

Training Efficient_VGG [NO_AUG] for up to 120 epochs...
Efficient_VGG [NO_AUG] -> Best Val: 0.7600 | Test: 0.7535

Training ResSE_AudioCNN [NO_AUG] for up to 120 epochs...
Efficient_VGG [NO_AUG] -> Best Val: 0.7600 | Test: 0.7535

Training ResSE_AudioCNN [NO_AUG] for up to 120 epochs...
ResSE_AudioCNN [NO_AUG] -> Best Val: 0.8195 | Test: 0.7990
ResSE_AudioCNN [NO_AUG] -> Best Val: 0.8195 | Test: 0.7990

Training UNet_Audio_Classifier [NO_AUG] for up to 120 epochs...

Training UNet_Audio_Classifier [NO_AUG] for up to 120 epochs...
UNet_Audio_Classifier [NO_AUG] -> Best Val: 0.8440 | Test: 0.8180
Saved: /home/alepot55/Desktop/projects/naml_project/reports/training_summary
_NO_AUG.csv
```

	Model	Tag	Best_Val_Accuracy	Test_Accuracy	Epochs_Run	Para
0	Efficient_VGG	WITH_AUG	0.7530	0.7520	96	344
1	ResSE_AudioCNN	WITH_AUG	0.8155	0.7980	55	12327
2	UNet_Audio_Classifier	WITH_AUG	0.8525	0.8230	73	11761
3	Efficient_VGG	NO_AUG	0.7600	0.7535	81	344
4	ResSE_AudioCNN	NO_AUG	0.8195	0.7990	61	12327
5	UNet_Audio_Classifier	NO_AUG	0.8440	0.8180	63	11761